TD 02 - Extra Time - Sujet et consignes

Ce TD est réalisé sous la forme d'un atelier afin de vous placer en situation professionnelle.

1. Sujet

Extra Time est une société fictive spécialisée dans la collecte et la diffusion en temps réel d'informations relatives aux matchs de football.

Elle met ces informations à la disposition de plateformes partenaires, leur permettant de proposer à leurs utilisateurs un suivi en direct des rencontres et de l'évolution des scores.

Dans le cadre du développement de ses activités, **Extra Time** souhaite faire concevoir et réaliser un nouveau service **back-end exposée par une API REST dédiée**.

Les besoins fonctionnels et les contraintes associés à ce projet ne sont volontairement pas détaillés dans le présent document. Votre équipe désigne un Product Owner chargé de recueillir la demande.

La formalisation du besoin, puis la conception, le développement, les tests et la documentation sont effectués de façon collective.

2. Objectifs pédagogiques

L'objectif est de réinvestir les apprentissages de la formation sur un projet moins guidé. Vous devez appliquer une démarche de qualité logicielle guidée sur les tests et organiser les vérifications selon la pyramide des tests.

Le TD évalue autant votre démarche que le résultat obtenu :

- pertinence des questions et des reformulations ;
- traçabilité des exigences et des décisions ;
- capacité à identifier les scénarios clés et prioriser la réalisation en conséquence ;
- qualité de la conception, de l'implémentation et des tests ;
- capacité à justifier les limites de la solution.

3. Organisation

- **Équipe :** binôme.
- **Stack technologique :** libre, sous réserve de validation avant le développement.
- **Paradigmes de programmation** : déclaratif et fonctionnel recommandés (fonctions pures, immutabilité, absence d'effet de bord…).
- **Durée :** environ 9 heures, réparties sur les jours 2 et 3 de la formation.

La feuille de route fournie séparément précise l'ordre des séquences et les productions intermédiaires attendues.

4. Productions attendues

- préparation de l'entretien et recueil du besoin auprès de l'expert métier ;
- formalisation du besoin et des règles métier ;
- conception du domaine adaptée au besoin retenu ;

- stratégie de vérification définie avant l'implémentation ;
- API REST et sa couche de persistance ;
- tests organisés selon la pyramide des tests :
  - tests unitaires (logique métier) ;
  - tests d'intégration ciblés avec le stockage réel ;
  - parcours API automatisé par une séquence de requêtes HTTP, avec Bruno dans la stack de référence ;
- documentation nécessaire à la compréhension et à l'exécution du projet.

Le raccordement au composant SSE fourni est facultatif. Son absence n'est pas pénalisée.

5. Livrables

Rendu via un **dépôt GitHub privé**. Ajouter **shrp777** / alex@shrp.dev en tant que collaborateur du projet.

1. Code source de l'application et des tests ;
2. Consignes et éléments nécessaires pour exécuter le projet ;
3. Document synthétique de 4 pages maximum, sans code ni copie brute de sorties d'outils, contenant :
   - questions principales, exigences, décisions et ambiguïtés restantes ;
   - modèle du domaine, états, transitions et invariants retenus ;
   - risques, cas de test, frontières, oracles et niveaux de test ;
   - preuves obtenues, limites, risque résiduel, difficultés rencontrées et, le cas échéant, retour sur l'usage de l'IA générative.

6. Usage de l'IA générative

L'IA générative peut être utilisée comme aide à la compréhension et à la vérification.

- **Autorisé** : demander une explication, rechercher la cause d'une erreur après une première tentative, comparer des solutions ou faire relire un extrait écrit par vos soins.
- **Interdit** : faire générer ou modifier une fonctionnalité demandée, intégrer du code non compris ou non vérifié, ou transmettre une information sensible.
- **Déclaration** : si vous utilisez une IA générative, indiquez dans le fichier `README.md` les usages ayant influencé votre réalisation, les propositions retenues et les vérifications effectuées.

Vous devez pouvoir expliquer et modifier tout élément remis. Une partie non maîtrisée ou un usage significatif non déclaré ne pourra pas être considéré comme acquis lors de l'évaluation.

7. Critères d'évaluation

- Démarche :
  - recueil du besoin et formalisation des exigences ;
  - analyse, conception et modélisation ;
  - préparation des vérifications avant l'implémentation.
- Réalisation :
  - conformité fonctionnelle et technique ;

  - qualité du code : nommage, structure, lisibilité, typage et séparation des responsabilités logicielles.
- Pertinence et répartition des tests dans la pyramide.
- Qualité de la documentation.
- Régularité du travail et pertinence des commits.